

用于 Branch-and-Bound Upper-Bound Rollout 的快速子节点选择复合指标

面向多机器人任务分配：仅使用当前子任务与剩余必做任务集合，不估算后续顺序或下界

最终建议

对每个候选 child c ，先计算其“当前子任务 + 后续全部必做子任务”的总归一化资源工作量 $W(c)$ ；优先选择 $W(c)$ 最小的 child。若 $W(c)$ 相同，再选择当前子任务归一化资源需求 $r(a_c)$ 最大的 child。

$$K(c) = (W(c), -r(a_c)), \quad c^* = \arg \min_{\text{lex}} K(c)$$

该规则只生成一次 rollout，运行时可做到 $O(|\text{Child}(v)|)$ ，不计算机器人旅行时间、不试探分配、不估算后续执行顺序，也不承担 lower bound 的证明功能。

1. 问题背景与设计约束

在 upper-bound heuristic 中，当前 plan-graph 节点可能有多个子节点。每个 child 对应一个可立即选择的子任务；选择之后，算法继续进行贪婪机器人分配，直到得到完整可行方案。

该选择问题具有两个层面：一方面，不同 child 可能通向不同的剩余必做任务集合；另一方面，当多个 child 实际上只是改变同一组任务的执行顺序时，应优先处理资源压力更大的当前任务。

本设计遵循三个约束：只执行一次 rollout；不为每个 child 计算类似 lower bound 的未来代价；只利用当前子任务的多资源需求，以及选择该 child 后能够确定的剩余必做任务集合。

已知信息	不要求的信息
child 对应的下一个子任务	后续必做任务的具体执行顺序
子任务对各机器人类型的数量需求	每个 child 的试探性机器人分配
选择 child 后的剩余必做任务集合	旅行时间、最早完成时间或未来 makespan
各机器人类型的系统总数量	新的 lower bound 或路径估价

2. 文献启发与适用边界

Brucker 等人的 RCPSP upper-bound heuristic 使用容量归一化的资源权重：

$$b_i = \sum_k (r_{ik} / b_k)$$

其中 r_{ik} 是活动 i 对资源 k 的需求， b_k 是资源容量。该 heuristic 在时序可执行的候选范围内，优先选择资源权重较大的活动，并将其安排在最早可行时间 [1]。

Liu 等人的带转移时间项目调度算法则采用 LST、LFT、GRD、WRUP、LPT、MTS 与 GRPW 等快速 priority rules，从 eligible activities 中逐步构造完整可行调度 [2]。这说明 upper-bound rollout 的活动选择可以使用计算廉价、问题相关的优先指标，而不必等同于 lower-bound 估计。

关键区别

若所有任务最终都必须执行，资源压力大的活动可以优先处理；但若不同 child 对应不同的 continuation，则不能只选择当前资源需求最大的 child，因为那可能主动进入总任务更多、总资源工作量更大的路径。

3. 指标定义

3.1 单个子任务的归一化资源需求

设机器人类型集合为 K 。对于子任务 a ：

- $d_{\{a,k\}}$ ：子任务 a 同时需要的 k 型机器人数量；
- N_k ：系统中 k 型机器人的总数量。

$$r(a) = \sum_{\{k \in K\}} d_{\{a,k\}} / N_k$$

该值把多种机器人资源统一压缩为一个标量，并通过 N_k 体现类型稀缺性。子任务同时涉及多种资源时，各类型需求均进入求和。

3.2 child 的总必做资源工作量

对候选 child c ，记：

- a_c ：child c 当前对应的下一个子任务；
- $M(c)$ ：执行 a_c 后仍然必须执行的子任务集合；
- $Q(c) = \{a_c\} \cup M(c)$ ：选择 c 后需要承担的全部必做任务集合。

$$W(c) = \sum_{\{a \in Q(c)\}} r(a) = \sum_{\{a \in Q(c)\}} \sum_{\{k \in K\}} d_{\{a,k\}} / N_k$$

$W(c)$ 只统计静态的必做资源工作量，不使用后续任务顺序、机器人当前位置、旅行时间或同步完成时间，因此它不是 lower bound，也不试图预测真实 makespan。

3.3 最终复合优先级

$$K(c) = (W(c), -r(a_c))$$

$$c^* = \arg \min_{\{c \in \text{Child}(v)\}} \text{lex } K(c)$$

按字典序选择意味着：

第一层 - continuation 选择：优先选择 $W(c)$ 最小的 child，即选择总必做归一化资源工作量更小的 continuation。

第二层 - 当前顺序选择：仅当多个 child 的 $W(c)$ 相同时，选择 $r(a_c)$ 最大的 child，即优先执行当前资源压力更大的子任务。

4. 为什么该指标能统一不同情形

情形	指标如何退化	选择结果
不同 child 的必做任务集合不同	$W(c)$ 通常不同	选择总必做资源工作量最小的 continuation
各 child 包含完全相同的必做任务集合，只是当前顺序不同	所有 $W(c)$ 相同	自动退化为选择当前 $r(a_c)$ 最大的 child
当前子任务资源需求相同，但后续必做集合不同	第二分量相同	直接按 $W(c)$ 选择整体更轻的 continuation
$W(c)$ 与 $r(a_c)$ 均相同	指标完全相同	用固定 child ID 或生成顺序确定性破平局

5. 示例

假设系统包含两类机器人： $N_A = 4$ ， $N_B = 2$ 。单个子任务的资源权重为：

$$r(a) = d_{\{a,A\}}/4 + d_{\{a,B\}}/2$$

候选 child	当前任务 $r(a_c)$	后续必做任务权重和	$W(c)$	$K(c)$
c_1	1.0	3.0	4.0	(4.0, -1.0)
c_2	0.6	2.2	2.8	(2.8, -0.6)

因为 $2.8 < 4.0$ ，选择 c_2 。即使 c_1 当前资源压力更大，也不应主动进入整体必做资源工作量更大的 continuation。

若两个 child 对应完全相同的任务集合，则 $W(c_1) = W(c_2)$ 。此时比较第二分量 $-r(a_c)$ ，等价于选择当前资源需求最大的任务，恢复 Brucker-style 的活动排序逻辑。

6. 算法流程与伪代码

建议将每个子任务的 $r(a)$ 与每个 child 的 $W(c)$ 离线预计算。upper-bound rollout 运行时只需读取两个数并进行字典序比较。

```

SelectChildFast(v):
    best_child <- null
    best_key   <- (+infinity, +infinity)

    for each feasible child c of v:
        current_demand <- r(a_c)
        total_workload <- W(c)
        key <- (total_workload, -current_demand)

```

```

    if key is lexicographically smaller than best_key:
        best_key    <- key
        best_child  <- c

    else if key = best_key and ID(c) < ID(best_child):
        best_child  <- c

return best_child

```

7. 计算复杂度与预计算

直接计算时，若 child c 的必做任务集合大小为 $|M(c)|$ ，机器人类型数为 $|K|$ ，则一次节点选择的代价为：

$$O(\sum_c |Q(c)| \cdot |K|)$$

更推荐在 plan graph 构造完成后预计算 $r(a)$ 和 $W(c)$ 。此时 rollout 中每个 child 仅需常数时间读取，节点选择复杂度下降为：

$$O(|\text{Child}(v)|)$$

需要执行	不需要执行
✓ 读取 $W(c)$	× 为每个 child 试探分配
✓ 读取 $r(a_c)$	× 计算旅行时间
✓ 字典序比较	× 估算后续任务顺序
✓ 选中后只对一个 child 做正式贪婪分配	× 计算新的 lower bound
	× 多次 rollout

8. 与 upper bound 正确性的关系

- $K(c)$ 是 rollout priority，不是 lower bound，也不能用于证明某个 BnB 节点可以安全剪枝。
- 该指标可以是非 admissible 的；它只影响可行解被找到的速度和质量，不影响 BnB 穷举与剪枝规则本身的正确性。
- 只有当 rollout 最终生成并验证了一个完整可行方案时，该方案的 makespan 才能用于更新全局 upper bound。若 rollout 失败，则本次 heuristic 不更新 incumbent。

9. 建议用于论文的表述

中文版本

对于当前节点的每个可行子节点 c ，记 a_c 为其对应的下一个子任务， $M(c)$ 为选择 c 后仍须执行的子任务集合。首先定义子任务 a 的归一化机器人需求 $r(a) = \sum_{\{k \in K\}} d_{\{a,k\}} / N_k$ ，其中 $d_{\{a,k\}}$ 是子任务对 k 型机器人的同时需求， N_k 是系统中该类型机器人的总数。进一步定义选择 c 后的总必做资源工作量

$W(c) = \sum_{a \in \{a_c\} \cup M(c)} r(a)$ 。upper-bound rollout 使用复合优先级 $K(c) = (W(c), -r(a_c))$ ，并选择字典序最小的子节点。该规则首先偏向总必做资源工作量较小的 continuation；当总工作量相同时，优先执行当前归一化资源需求较大的子任务。

English version

For each feasible outgoing child c , let a_c denote its immediate subtask and $M(c)$ the set of mandatory subtasks remaining after selecting c . The normalized robot demand of a subtask a is defined as $r(a) = \sum_{k \in K} d_{\{a, k\}} / N_k$, where $d_{\{a, k\}}$ is the simultaneous demand for type- k robots and N_k is the available number of type- k robots. The total mandatory resource workload associated with child c is $W(c) = \sum_{a \in \{a_c\} \cup M(c)} r(a)$. The upper-bound rollout selects the child with the lexicographically smallest priority key $K(c) = (W(c), -r(a_c))$. Hence, the rule first favors continuations with a smaller total mandatory resource workload and, when the workloads are equal, prioritizes the immediate subtask with the greater normalized robot demand.

10. 设计结论

推荐的最终规则

$$K(c) = (W(c), -r(a_c)), \quad c^* = \arg \min_{\text{lex}} K(c)$$

它用一个无参数的字典序复合 key 同时处理“不同 continuation 的任务量选择”和“相同任务集合下的执行顺序选择”，并满足单次 rollout、快速计算和不引入 LB 式估计的要求。

参考文献

- [1] P. Brucker, S. Knust, A. Schoo, and O. Thiele, “A branch and bound algorithm for the resource-constrained project scheduling problem,” *European Journal of Operational Research*, vol. 107, pp. 272-288, 1998.
- [2] Y. Liu, S. Jin, J. Zhou, and Q. Hu, “A branch-and-bound algorithm for the unit-capacity resource constrained project scheduling problem with transfer times,” *Computers & Operations Research*, vol. 151, 106097, 2023.
- [3] D. R. Morrison, S. H. Jacobson, J. J. Sauppe, and E. C. Sewell, “Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning,” *Discrete Optimization*, vol. 19, pp. 79-102, 2016.